

Case studies

This page shows how to use local themes to customize various aspects of notebooks and websites in PDF or HTML. Fully reproducible examples for each case study are available on Github:

- Case study files

Tufte

This case study shows how to build a local theme on top of academic to replicate the Tufte CSS article style: serif typography, warm paper colors, restrained accents, sidenotes, margin figures, and code/output surfaces that match the page.

The source tree is intentionally small:

```
project/  
  tufte.typ  
  theme/  
    theme.toml  
  css/  
    tufte.css
```

The theme directory itself declares its base theme:

```
# theme/theme.toml  
extends = "academic"
```

That keeps all of the built-in academic theme structure: the single-document HTML wrapper, the theme toggle, sidenotes, side figures, code styling, and dark-mode support. The local `tufte/css/tufte.css` file is intentionally small. It overrides the public `--calepin-*` tokens instead of targeting private theme internals:

```
:root, html[data-theme="light"] {  
  --calepin-font-body: Palatino, "Palatino Linotype", "Book Antiqua", Georgia, serif;  
  --calepin-font-heading: Palatino, "Palatino Linotype", "Book Antiqua", Georgia, serif;  
  --calepin-surface-code: #fffd0;  
  --calepin-surface-output: #fffd0;  
  --calepin-surface: #fffd0;  
  --calepin-surface-muted: #fff8eb;  
  --calepin-color-background: #fffff8;  
  --calepin-color-text: #12110f;  
  --calepin-color-muted: #5a5650;  
  --calepin-color-accent: #7a2e2a;  
  --calepin-color-accent-hover: #5f2520;  
  --calepin-color-link: #7a2e2a;  
  --calepin-color-link-hover: #5f2520;  
}  
  
html[data-theme="dark"] {  
  --calepin-color-background: #18130d;  
  --calepin-color-text: #f6efe8;  
  --calepin-color-muted: #c5baa7;  
  --calepin-color-accent: #d58a7f;  
  --calepin-color-accent-hover: #f0b7ab;  
  --calepin-color-link: #d58a7f;  
  --calepin-color-link-hover: #f0b7ab;  
  --calepin-surface-code: #1f1a14;  
  --calepin-surface-output: #1f1a14;  
  --calepin-surface: #1f1a14;  
  --calepin-surface-muted: #2b241b;  
}
```

The document source can then use the normal academic-theme elements:

`calepin.elements.sidenote` for margin notes, `calepin.elements.sidefigure` for margin figures, and regular executable code chunks for computed output. The stylesheet changes the feel of those elements, but the layout behavior still comes from the built-in theme.

From the project root, render the HTML and PDF with:

```
calepin compile tufte.typ --format html --set theme=./theme  
calepin compile tufte.typ --format pdf --set theme=./theme
```

The theme path is resolved from the project root when no `--config` file is provided. Keeping the local theme and document together makes the example portable: copy the directory, run the same commands, and the academic theme plus Tufte overlay are applied in both rendered outputs.

Classicthesis

This case study shows how to use a local theme with `layouts/pdf.typ` to wrap a Typst document in the `classicthesis` template.

The project can stay small:

```
project/  
book.typ  
theme/  
  theme.toml  
layouts/  
pdf.typ
```

The theme manifest disables inherited Calepin styling so the PDF layout comes entirely from the template:

```
# theme/theme.toml  
extends = "typst"
```

First put the `classicthesis` MiniJinja template in `theme/layouts/pdf.typ`:

```
#import "@preview/classicthesis:0.1.0": *  
  
#show: classicthesis.with(  
  title: "My Book Title",  
  subtitle: "A Subtitle",  
  author: "Author Name",  
  date: "2025",  
  dedication: [To my readers.],  
  abstract: [This book explores...],  
)  
  
{{ doc.body }}
```

Then write the notebook content without the template preamble:

```
= Chapter One  
  
== Section  
  
More content...
```

Here, the `classicthesis` template provides the page design and chapter structure, while the notebook body still comes from your `.typ` file. `doc.body` is where Calepin injects the document source.

From the project root, render the PDF with:

```
calepin compile book.typ --format pdf --set theme=./theme
```

If you want the same document to use a different PDF layout later, swap the contents of `theme/layouts/pdf.typ` without changing the document itself.

Website fonts

This case study shows how to override a website's fonts using Google Fonts API. We start by creating an example website scaffold based on the default calepin theme. The following command will create a new directory called `calepin_website`:

```
calepin new website --theme calepin
```

Then, we add simple local theme directory with a manifest `theme.toml` and a single `fonts.css` file:

```
calepin_website/  
  
theme/      # theme directory  
  theme.toml  
  css/  
    fonts.css  
  
calepin.toml # website configuration file  
  
index.typ   # website content  
404.typ  
assets/  
posts/
```

The theme manifest, `calepin_website/theme/theme.toml`, inherits from the built-in `calepin` theme:

```
extends = "calepin"
```

We add a single CSS file which imports Google Fonts and overrides the public font tokens:

```
@import url("https://fonts.googleapis.com/css2?family=Rubik+Moonrocks&family=Space+Grotesk:wght@400;500;700&family=IBM+Plex+Mono:wght@400;500&display=swap");  
  
:root {  
  --calepin-font-body: "Space Grotesk", system-ui, sans-serif;  
  --calepin-font-heading: "Rubik Moonrocks", "Space Grotesk", sans-serif;  
  --calepin-font-mono: "IBM Plex Mono", ui-monospace, monospace;  
}
```

This keeps the layout, navigation, and page behavior from the built-in `calepin` theme. The local theme only changes typography, so the result is easy to reason about: same site structure, different font stack.

Finally, we render and serve the website:

```
calepin compile calepin_website --set theme=./theme  
calepin serve calepin_website --open
```